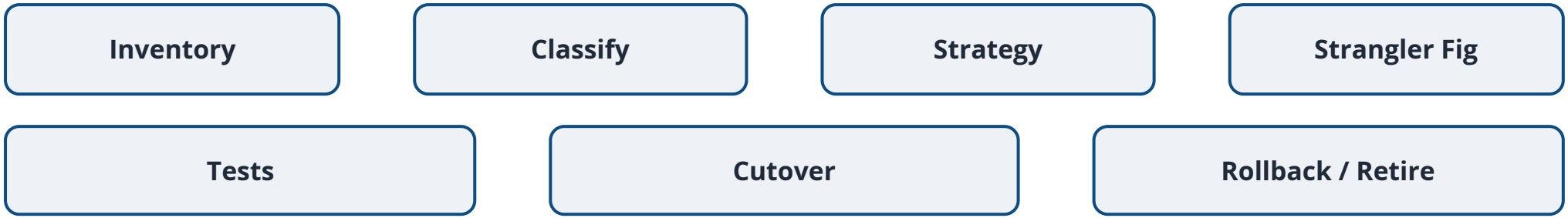
Legacy Migration Playbook

From inventory to cutover, with rollback at every step.
Generic method based on public practices · Fictional examples

pm-toolkit · Portfolio material · No client information

1. Seven phases, one rule: small and reversible



Migrate in waves, never in one big bang.

Every wave has an owner, an exit criterion and a way back.

Business keeps shipping features while the migration runs.

2. Inventory: know what you have

Collect

- Applications, modules, owners
- Runtime, framework, library versions
- Databases, jobs, file shares
- Integrations and contracts
- Users, volume, peak periods

Sources

- Repositories and build files
- Dependency and vulnerability scans
- Logs and monitoring data
- Interviews with the people who run it

Output: one register, one row per component.

3. Classify: value vs. health

Business value	Technical health	Typical decision
High	Poor	**Modernize first** (refactor or rewrite)
High	Good	Keep and maintain
Low	Poor	Retire or replace with a product
Low	Good	Leave alone, monitor

Also score: security exposure, end of support, change frequency, team knowledge, dependencies on other components.

4. Strategy: pick one per component

Option	What it means	Use when	Watch out
Rehost	Move as is to new infrastructure	Deadline on hardware or data center	Debt moves with it
Replatform	Small changes to use a managed service	Cheap wins, e.g. managed database	Scope creep
Refactor	Change code or runtime, keep behavior	Logic is valuable, stack is outdated	Needs tests first
Rewrite	Rebuild with new design	Logic is poor or well understood	Longest, riskiest
Retire / Buy	Switch off or use a product	Low value, commodity need	Data and users to migrate

5. Strangler Fig: replace piece by piece



The routing layer decides, per page or endpoint, who answers.

Start with a low-risk, well-bounded module.

Rolling back = changing the route.

Plan from day one: shared login and shared state across both stacks.

Each legacy module gets a retirement date.

6. Tests: prove behavior did not change

Before touching code

- **Characterization tests:** record what the system does today, right or wrong
- Anonymized production-like data

During migration

- Unit and contract tests on new modules
- **Parallel run:** old and new answer the same input; compare outputs
- Performance baseline vs. new

Gate

- Coverage gate in the pipeline
- Zero critical differences in the parallel run
- Security and dependency scan green

7. Cutover: gradual, with a go/no-go

- **Prepare** Freeze scope, announce the window, confirm rollback works in staging.
- **Pilot** Move a small group (one country, one team, 5% of traffic).
- **Expand** Canary release, then waves, using feature toggles or routing.
- **Hypercare** Extra monitoring and a support rota for 1–2 release cycles.
- **Retire** Switch off the legacy module after a stable period; archive data.

Go/no-go checklist: tests green · data migrated and reconciled · monitoring ready · rollback tested · support briefed · sponsor approval.

8. Rollback: decide before you need it

Trigger (examples)	Action
Error rate above agreed threshold after release	Route traffic back to legacy
Data mismatch found in reconciliation	Stop writes to new module, reconcile, retry
Critical business flow degraded	Disable feature toggle, open incident

Rollback owner and decision time limit are named in advance.

Database changes must be backward compatible until the retirement date.

Rehearse the rollback at least once before production.

9. Measure and govern

Delivery

- Release frequency
- Lead time to production
- Velocity and commitment reliability

Quality

- Incidents per release
- Open bugs and escaped defects
- Test coverage on migrated modules

Progress

- % of components migrated
- Debt hours closed vs. planned
- Legacy servers and licenses retired

Take a baseline before wave 1. Review in a monthly steering committee; re-score after each wave.